RESIDÊNCIA TECNOLÓGICA SELDON

Manual Técnico-Científico & Pedagógico

Formação Avançada em Android AOSP, Inteligência Artificial, Cibersegurança Móvel e Otimização Energética

Estrutura Curricular Integral (435 Horas)

Módulo 1:	Núcleo Comum de Formação	210h (7 Disciplinas)
Módulo 2:	Trilha de Inteligência Artificial	75h (3 Disciplinas)
Módulo 3:	Trilha de Segurança de Dispositivos Móveis	75h (3 Disciplinas)
Módulo 4:	Trilha de Eficiência Energética	75h (3 Disciplinas)

Documento de Referência Curricular e Técnica
Baseado no Programa de Pós-Graduação e Formação Tecnológica Seldon

Outubro de 2026

Sumário

1 Módulo 1 – Núcleo Comum de Formação (210 Horas) 3

1.1 Dados, Experimentação e Metodologia Científica (30h) . . . 3

1.1.1 Formulação de Problemas Tecnológicos e Hipóteses . . . 3

1.1.2 Desenho Experimental e Variáveis de Controle . . . 4

1.1.3 Análise Estatística Inferencial: Teste t de Student e ANOVA . . . 4

1.2 Engenharia de Software, Requisitos e Gestão do Desenvolvimento (24h) . . . 5

1.2.1 Engenharia de Requisitos Funcionais e Não Funcionais . . . 5

1.2.2 Matriz de Rastreabilidade de Requisitos (RTM) . . . 5

1.3 Metodologia de Projetos, Ética e Habilidades Profissionais (30h) . . . 6

1.3.1 Aprendizagem Baseada em Projetos (PBL) na Residência . . . 6

1.3.2 Governança de Dados e Conformidade com a LGPD . . . 6

1.4 Arquitetura de Software (18h) . . . 6

1.4.1 Clean Architecture no Ecossistema Android . . . 7

1.4.2 Implementação Prática: UseCase Puro com Kotlin . . . 7

1.5 Inteligência Artificial: Fundamentos, Frameworks e Aplicações (36h) . . . 8

1.5.1 O Pipeline Clássico de Aprendizado Supervisionado . . . 8

1.5.2 Matriz de Confusão e Métricas de Validação Rigorosas . . . 8

1.6 Programação Java/Kotlin e Desenvolvimento de Aplicações Android (42h) . . . 8

1.6.1 Os Quatro Componentes Canônicos do AOSP . . . 8

1.6.2 Interface Reativa com Jetpack Compose e StateFlow . . . 9

1.7 Testes, Automação, Segurança e Integração Contínua (30h) . . . 10

1.7.1 A Pirâmide de Testes no Android . . . 10

1.7.2 Pipeline CI/CD Automatizado com GitHub Actions . . . 10

2 Módulo 2 – Trilha de IA aplicada à Otimização de Sistemas Android (75 Horas) 11

2.1 AOSP – Observabilidade e Desempenho (25h) . . . 11

2.1.1 A Arquitetura em Camadas do AOSP . . . 11

2.1.2 O Mecanismo de Comunicação Inter-Processos: Binder IPC . . . 12

2.1.3 Observabilidade em Nanossegundos com Perfetto Trace . . . 12

2.2 Aprendizado de Máquina e Aprendizado Profundo (30h) . . . 13

2.2.1 Detecção de Anomalias com Autoencoders Neurais . . . 13

2.3 IA On-device e Otimização de Modelos (20h) . . . 14

2.3.1 Teoria Matemática da Quantização Afim (FP32 $\rightarrow$ INT8) . . . 14

2.4 Revisão da Literatura e Estado da Arte: IA e Observabilidade em SO . . . 14

2.5 Arquitetura de Engenharia do Projeto Final: Seldon Anomaly Watcher . . . 15

2.5.1 Especificação Técnica dos Módulos da Solução . . . 16

3 Módulo 3 – Trilha de Segurança de Dispositivos Móveis e Automação (75 Horas) 17
3.1 Introdução à Cibersegurança, Modelagem de Ameaças e Vulnerabilidades (35h) 17
3.1.1 Metodologia de Modelagem de Ameaças: STRIDE 17
3.1.2 Superfícies de Ataque Críticas no Android 17
3.2 AOSP – Arquitetura de Segurança Android (20h) 18
3.2.1 A Sandbox Baseada em UID Unix 18
3.2.2 Security-Enhanced Linux (SELinux): Mandatory Access Control . . . 18
3.2.3 Android Keystore & Hardware StrongBox 19
3.3 Automação de Testes e Validações de Segurança (20h) 19
3.3.1 Integração DevSecOps Contínua 19
3.4 Revisão da Literatura e Estado da Arte em Cibersegurança Móvel 20
3.5 Arquitetura de Engenharia do Projeto Final: Seldon SecGuard DevSecOps . . 20
3.5.1 Especificação Técnica dos Módulos do Sistema 20

4 Módulo 4 – Trilha de Eficiência Energética e Algoritmos de Otimização (75 Horas) 22
4.1 AOSP – Gestão Energética da Plataforma (25h) 22
4.1.1 O Subsistema PowerManagerService e os WakeLocks 22
4.1.2 A Máquina de Estados do Doze Mode 23
4.2 Medição e Profiling Energético (25h) 23
4.2.1 Fundamentos Físicos da Potência e Energia 23
4.2.2 Instrumentação Física em Bancada com o Monsoon Power Monitor . . 24
4.3 Modelagem e Algoritmos de Otimização (25h) 24
4.3.1 Teoria da Otimização Multiobjetivo e Fronteira de Pareto 24
4.3.2 O Algoritmo Genético NSGA-II 24
4.4 Revisão da Literatura e Estado da Arte em Eficiência Energética Móvel . . . 25
4.5 Arquitetura de Engenharia do Projeto Final: Seldon EcoOptimizer 26
4.5.1 Especificação Técnica dos Módulos do Sistema 26

5 Ambiente de Engenharia AOSP Android 15 e Política de Documentação 28
5.1 Fundamentação da Árvore AOSP Android 15 28
5.1.1 A Interface de IPC: AIDL NDK e Estabilidade VINTF 29
5.2 O Daemon Nativo de Sistema (`seldond`) 29
5.3 Confinamento de Segurança: Políticas SELinux 30
5.4 Política de Engenharia: Documentação Obrigatória por Feature 30
5.5 Roteiro de Compilação e Emulação AOSP 31

Capítulo 1

Módulo 1 – Núcleo Comum de Formação (210 Horas)

O **Núcleo Comum de Formação** constitui a base obrigatória e estruturante do programa de Residência Tecnológica Seldon. Seu propósito central é nivelar os residentes em fundamentos sólidos de engenharia de software, rigor do método científico, arquitetura de sistemas móveis, desenvolvimento Android com Kotlin moderno, testes automatizados e gestão ágil de projetos orientada a evidências.

Composto por sete disciplinas complementares que totalizam 210 horas letivas, este módulo estabelece o vocabulário conceitual e a capacitação técnica essenciais para a atuação nas três trilhas de especialização posteriores.

—

1.1 Dados, Experimentação e Metodologia Científica (30h)

Objetivo Pedagógico da Disciplina

Capacitar os residentes para formular problemas de engenharia tecnológica com rigor formal, desenhar ensaios experimentais reprodutíveis, estabelecer métricas objetivas de desempenho e interpretar evidências estatisticamente válidas, respeitando os preceitos de propriedade intelectual.

1.1.1 Formulação de Problemas Tecnológicos e Hipóteses

Na engenharia de software contemporânea, decisões arquiteturais e otimizações de baixo nível não podem se basear em intuição empírica desprovida de validação formal. O método científico aplicado a sistemas computacionais exige a estruturação de hipóteses estatísticas verificáveis:

- **Hipótese Nula (H_0):** Postula que a modificação introduzida (ex.: nova estratégia de compactação de memória ou novo algoritmo de renderização) não produz diferença estatisticamente significativa no parâmetro de interesse em comparação com a linha de base (*baseline*).
- **Hipótese Alternativa (H_1):** Postula que a intervenção tecnológica produz um efeito real mensurável (redução de latência, menor variabilidade temporal ou conservação de energia).

1.1.2 Desenho Experimental e Variáveis de Controle

O isolamento de ruído em plataformas móveis como o Android é um dos desafios mais complexos na coleta de métricas, devido a variáveis intervenientes não controladas:

1. **Variável Independente (X):** O fator sob manipulação direta do pesquisador (ex.: uso de `LazyColumn` em Jetpack Compose versus `RecyclerView` tradicional).
2. **Variável Dependente (Y):** A métrica de resposta observada (ex.: tempo de renderização por quadro T_{frame} em milissegundos).
3. **Variáveis de Controle e Ruído:** Fatores externos que devem ser congelados durante o ensaio:
   - Nível e temperatura da bateria (para evitar *thermal throttling*);
   - Frequência de clock da CPU fixada via `cpufreq` no kernel;
   - Brilho da tela fixado e rádio celular configurado em modo avião.

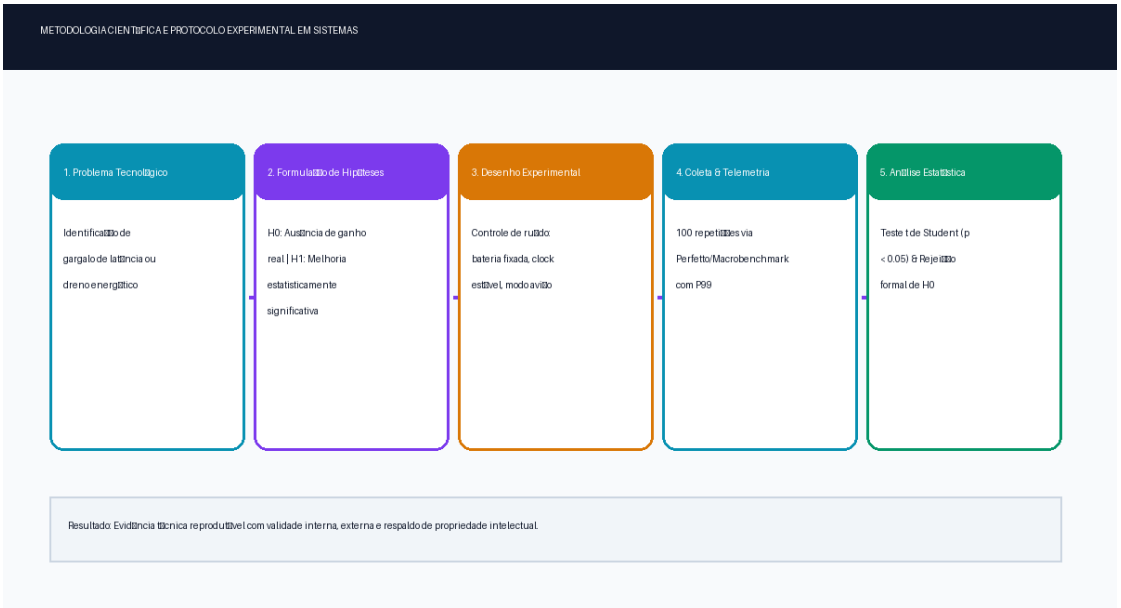


Figura 1.1: Fluxograma do Protocolo Experimental Rigoroso: formulação de hipóteses estatísticas, controle de ruído térmico/clock e validação via Teste t de Student ($p < 0{,}05$).

1.1.3 Análise Estatística Inferencial: Teste t de Student e ANOVA

Para rejeitar formalmente a hipótese nula H_0 com nível de significância $\alpha = 0{,}05$, calcula-se a estatística t para duas amostras independentes com variâncias populacionais desconhecidas:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \tag{1.1}$$

onde $\bar{X}_i$ representa a média amostral, s_i^2 a variância amostral e n_i o tamanho de cada lote de medição. A rejeição de H_0 ocorre quando o p-valor associado à distribuição t com os respectivos graus de liberdade for estritamente menor que α, comprovando que o ganho de desempenho não decorreu de mero acaso estocástico.

Caso Prático: Protocolo Experimental de Frame Jank

Cenário: Comprovar se uma alteração na alocação de objetos em um loop gráfico reduz a taxa de quadros engasgados (*jank frames*).
Protocolo: 100 repetições de um teste automatizado via `macrobenchmark`. Registro dos tempos no percentil 99 (P_{99}).
Resultado: $\bar{X}_{\text{antigo}} = 24{,}8$ ms, $\bar{X}_{\text{novo}} = 14{,}2$ ms. Estatística $t = 12{,}4$, resultando em $p < 0{,}001$. Conclusão: a hipótese nula é categoricamente rejeitada com 99,9% de confiança estatística.

—

1.2 Engenharia de Software, Requisitos e Gestão do Desenvolvimento (24h)

Objetivo Pedagógico da Disciplina

Desenvolver competências para elicitar, analisar e formalizar requisitos de software complexos, estruturar ciclos de desenvolvimento ágeis sustentáveis e assegurar a rastreabilidade bidirecional contínua entre o desafio industrial e a entrega executável.

1.2.1 Engenharia de Requisitos Funcionais e Não Funcionais

Em sistemas de missão crítica e plataformas embarcadas, a especificação precisa de requisitos constitui o primeiro baluarte contra falhas catastróficas:

- **Requisitos Funcionais (RF):** Descrevem os comportamentos intrínsecos do software (ex.: "O sistema deve sincronizar os dados biométricos com a nuvem a cada 15 minutos se houver conexão Wi-Fi").
- **Requisitos Não Funcionais (RNF):** Estabelecem restrições de qualidade, tais como:
  - Latência de resposta da interface gráfica $\leq 16{,}6$ ms em 60 Hz;
  - Consumo de corrente em repouso com tela apagada ≤ 15 mA;
  - Ocupação máxima em memória residente (RSS) ≤ 64 MB.

1.2.2 Matriz de Rastreabilidade de Requisitos (RTM)

A rastreabilidade bidirecional é implementada através de identificadores unívocos que conectam cada necessidade do projeto ao seu artefato de verificação:

$$\text{Desafio Industrial} \longrightarrow \text{RF/RNF} \longrightarrow \text{User Story} \longrightarrow \text{Branch/Commit} \longrightarrow \text{Teste Automatizado} \tag{1.2}$$

```
# Formato de mensagem de commit em conformidade com RTM:
feat(power): [RF-042][REQ-BAT-09] migra polling para WorkManager com
    restricao de carga

- Substitui ScheduledExecutorService por PeriodicWorkRequestBuilder
- Garante execucao exclusivamente quando o device estiver carregando e em Wi-
    Fi
- Evita dreno em background (RNF-ENG-03)
- Testes unitarios automatizados em WorkManagerTest.kt (Cobertura 100%)
```

Listing 1.1: Estrutura de Rastreabilidade em Commits Git

—

1.3 Metodologia de Projetos, Ética e Habilidades Profissionais (30h)

Objetivo Pedagógico da Disciplina

Preparar o residente para liderar e colaborar em equipes multidisciplinares sob a metodologia de Aprendizagem Baseada em Projetos (PBL), mantendo rigor ético profissional, governança de dados conforme a LGPD e documentação formal de decisões arquiteturais.

1.3.1 Aprendizagem Baseada em Projetos (PBL) na Residência

Diferente da pedagogia passiva tradicional, o PBL imerge os residentes em problemas reais trazidos por parceiros industriais. O ciclo de aprendizado estrutura-se em:

1. **Imersão e Delimitação:** Análise do problema e formulação de perguntas norteadoras.
2. **Planejamento e Decomposição:** Criação de *Product Backlog* e definição de papéis (Product Owner, Tech Lead, Especialista em QA).
3. **Ciclos de Prototipação e Experimentação:** *Sprints* quinzenais com geração de evidências técnicas tangíveis.
4. **Apresentação de Resultados & Retrospectiva:** Demonstrações técnicas para bancas de avaliação acadêmica e corporativa.

1.3.2 Governança de Dados e Conformidade com a LGPD

O desenvolvimento para dispositivos móveis envolve acesso a sensores sensíveis (GPS, câmera, microfone, identificadores de rádio). A Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018) impõe diretrizes estritas:

- **Privacy by Design:** Anonimização na borda através de funções de dispersão criptográfica (SHA-256 com *salt* local);
- **Minimização da Coleta:** Coleta estritamente restrita aos dados indispensáveis à execução do serviço contratado;
- **Relatório de Impacto à Proteção de Dados (RIPD):** Documentação formal descrevendo os fluxos de tratamento de dados e as salvaguardas tecnológicas implementadas.

—

1.4 Arquitetura de Software (18h)

Objetivo Pedagógico da Disciplina

Capacitar os residentes na aplicação de princípios SOLID, arquiteturas desacopladas (Clean Architecture, MVVM e MVI) e injeção de dependências em aplicações Android escaláveis e de alta testabilidade.

1.4.1 Clean Architecture no Ecossistema Android

A Clean Architecture preconiza a separação concêntrica de responsabilidades, assegurando que o núcleo de regras de negócio (*Domain*) permaneça agnóstico a detalhes de infraestrutura (bancos de dados, redes e frameworks de UI):

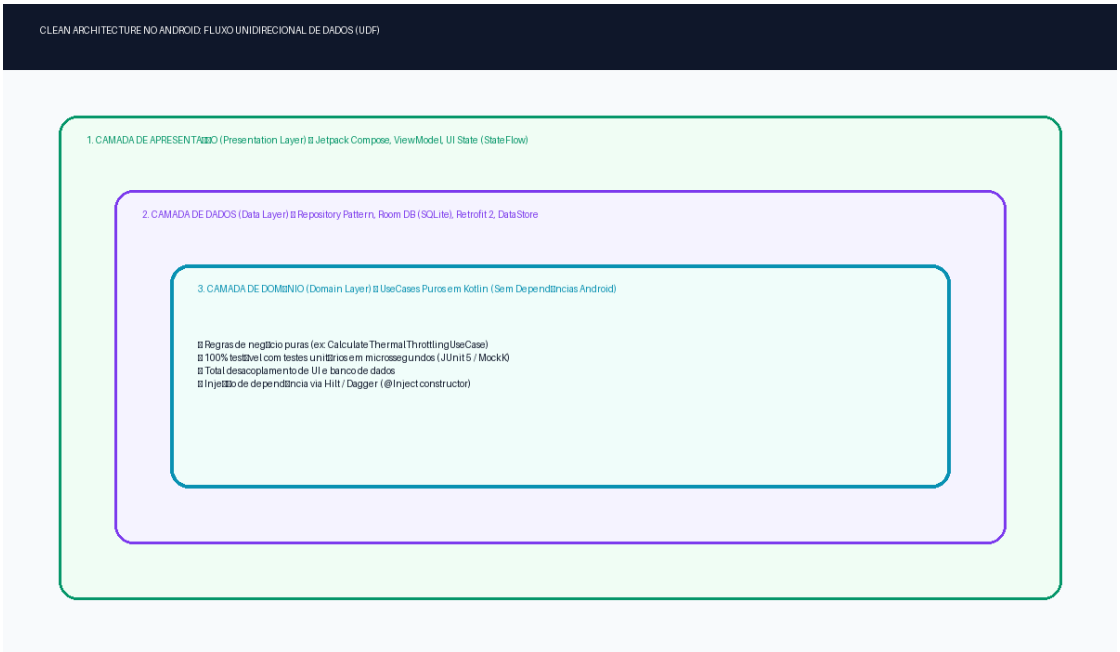


Figura 1.2: Diagrama de Camadas da Clean Architecture no Android: a camada de Domínio pura encapsulada pelas camadas de Dados e Apresentação com fluxo unidirecional.

1.4.2 Implementação Prática: UseCase Puro com Kotlin

```kotlin
package com.seldon.domain.usecase

import com.seldon.domain.model.ThermalThrottlingEvent
import com.seldon.domain.repository.SensorTelemetryRepository
import javax.inject.Inject

/**
 * Regra de negócio pura: avalia risco termico sem acoplamento ao Android SDK.
 */
class EvaluateThermalRiskUseCase @Inject constructor(
    private val telemetryRepository: SensorTelemetryRepository
) {
    suspend operator fun invoke(thresholdCelsius: Double): ThermalThrottlingEvent {
        val currentTemperature = telemetryRepository.readSocTemperature()
        val isCritical = currentTemperature >= thresholdCelsius

        return ThermalThrottlingEvent(
            currentTemperature = currentTemperature,
            threshold = thresholdCelsius,
            requiresIntervention = isCritical
        )
    }
}
```

Listing 1.2: Implementação de UseCase na Camada de Domínio

—

1.5 Inteligência Artificial: Fundamentos, Frameworks e Aplicações (36h)

Objetivo Pedagógico da Disciplina

Compreender a esteira completa de engenharia de Machine Learning e Deep Learning, desde a preparação de tensores de dados até o treinamento, validação cruzada, regularização e consumo de inferências preditivas integradas a aplicações móveis.

1.5.1 O Pipeline Clássico de Aprendizado Supervisionado

O aprendizado de máquina supervisionado visa estimar uma função $f : \mathcal{X} \to \mathcal{Y}$ que mapeie um vetor de características $\mathbf{x} \in \mathbb{R}^d$ em um rótulo $y \in \mathcal{Y}$. Em problemas de classificação, minimiza-se a função de perda de entropia cruzada (*Cross-Entropy Loss*):

$$\mathcal{L}_{\text{CE}}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{i,k} \log\left(\hat{y}_{i,k}(\mathbf{w})\right) \tag{1.3}$$

onde N é o número de amostras, K o número de classes e $\hat{y}_{i,k}$ a probabilidade predita pelo modelo parametrizado pelos pesos $\mathbf{w}$.

1.5.2 Matriz de Confusão e Métricas de Validação Rigorosas

Em sistemas operacionais, a ocorrência de anomalias (quedas bruscas de tensão ou travamentos) representa eventos raros (dados altamente desbalanceados). A acurácia global é inadequada; priorizam-se:

$$\text{Precision} = \frac{\text{VP}}{\text{VP} + \text{FP}}, \quad \text{Recall} = \frac{\text{VP}}{\text{VP} + \text{FN}}, \quad F_1\text{-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1.4}$$

—

1.6 Programação Java/Kotlin e Desenvolvimento de Aplicações Android (42h)

Objetivo Pedagógico da Disciplina

Capacitar os residentes na construção de aplicações Android modernas com Kotlin, arquitetura baseada em Coroutines e Flow, interface declarativa com Jetpack Compose e domínio profundo dos quatro componentes canônicos do AOSP.

1.6.1 Os Quatro Componentes Canônicos do AOSP

O ecossistema Android fundamenta-se em quatro pilares declarados no `AndroidManifest.xml`:

1. **Activity:** Representa uma interface gráfica com o usuário e controla o fluxo de vida visual (`onCreate`, `onStart`, `onResume`, etc.);
2. **Service:** Executa processamento contínuo em segundo plano sem interface gráfica (ex.: serviços de telemetria ou reprodução de áudio);

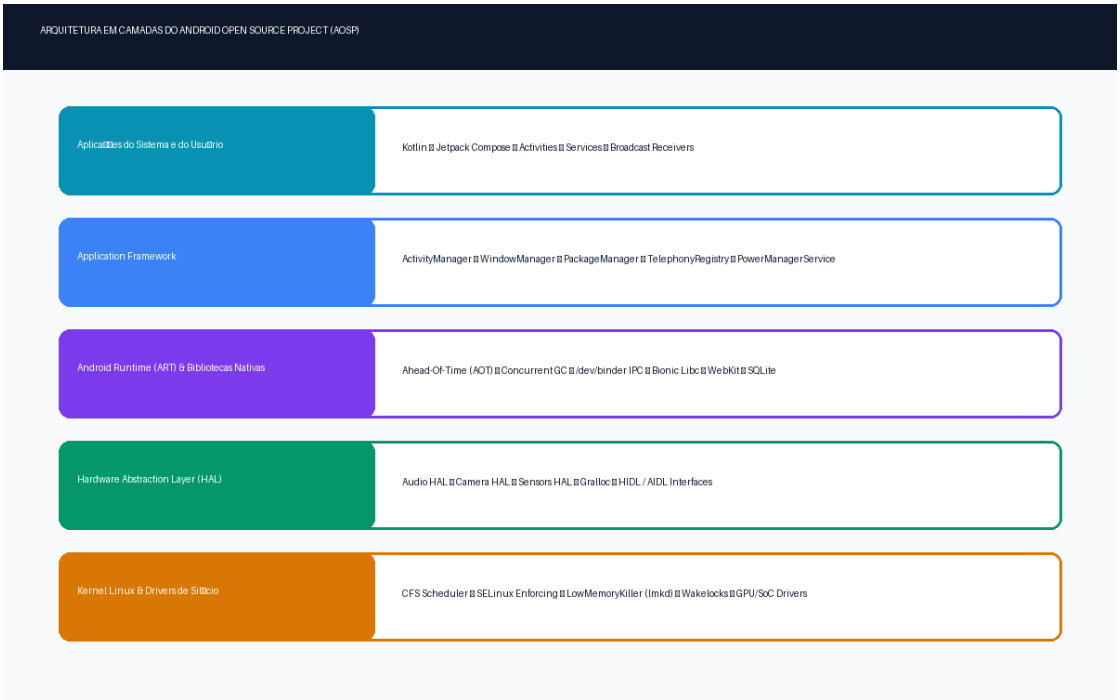


Figura 1.3: Visão Geral da Pilha Arquitetural do AOSP: desde o Kernel Linux e HAL até o Android Runtime (ART) e o Application Framework.

3. **Broadcast Receiver:** Permite à aplicação receber anúncios de eventos do sistema ou de outros apps (ex.: `ACTION_BATTERY_LOW`, `CONNECTIVITY_CHANGE`);

4. **Content Provider:** Gerencia o acesso seguro e estruturado a repositórios de dados compartilhados entre processos distintos.

1.6.2 Interface Reativa com Jetpack Compose e StateFlow

O Jetpack Compose adota o paradigma declarativo: a interface visual é uma função pura do estado atual $UI = f(State)$.

```
class TelemetryViewModel @Inject constructor(
    private val evaluateThermalRiskUseCase: EvaluateThermalRiskUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow<TelemetryUiState>(
        TelemetryUiState.Loading)
    val uiState: StateFlow<TelemetryUiState> = _uiState.asStateFlow()

    fun triggerAnalysis() {
        viewModelScope.launch {
            try {
                val result = evaluateThermalRiskUseCase(thresholdCelsius =
                    42.0)
                _uiState.value = TelemetryUiState.Success(result)
            } catch (e: Exception) {
                _uiState.value = TelemetryUiState.Error(e.localizedMessage)
            }
        }
    }
}
```

Listing 1.3: Exemplo de ViewModel reativa com StateFlow em Kotlin

—

1.7 Testes, Automação, Segurança e Integração Contínua (30h)

Objetivo Pedagógico da Disciplina

Compreender e implementar esteiras de automação de testes unitários, instrumentados e de integração contínua (CI/CD), aplicando análise estática de código e boas práticas de segurança defensiva.

1.7.1 A Pirâmide de Testes no Android

Uma estratégia eficiente de garantia de qualidade balanceia velocidade de execução com fidelidade de ambiente:

- **Testes Unitários (Base - 70%):** Executam na JVM local em milissegundos utilizando JUnit 5 e MockK. Validam regras de negócio puras;
- **Testes de Integração / Robolectric (Meio - 20%):** Simulam classes do SDK Android na JVM sem necessitar de emulador ou dispositivo físico;
- **Testes E2E Instrumentados (Topo - 10%):** Executam diretamente no dispositivo via Espresso ou UIAutomator, validando fluxos críticos de ponta a ponta.

1.7.2 Pipeline CI/CD Automatizado com GitHub Actions

```yaml
# Workflow de validacao continua no GitHub Actions:
name: Android CI Pipeline
on: [push, pull_request]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin' }
      - name: Analise Estatica (Detekt & Lint)
        run: ./gradlew detekt lintRelease
      - name: Testes Unitarios
        run: ./gradlew testReleaseUnitTest
      - name: Build do APK de Release
        run: ./gradlew assembleRelease
```

Listing 1.4: Etapas Críticas de uma Pipeline CI/CD Android

Capítulo 2

Módulo 2 – Trilha de IA aplicada à Otimização de Sistemas Android (75 Horas)

A **Trilha de Inteligência Artificial aplicada à Otimização de Sistemas Android** explora o coração do sistema operacional, focando no **Android Open Source Project (AOSP)**. Os residentes aprendem a extrair telemetria em nanossegundos do kernel Linux e do runtime ART, modelar padrões complexos de uso com redes neurais profundas e converter esses modelos em artefatos quantizados ultracompactos capazes de rodar em tempo real na NPU do dispositivo móvel com baixíssimo impacto energético.

—

2.1 AOSP – Observabilidade e Desempenho (25h)

Objetivo Pedagógico da Disciplina

Compreender a arquitetura interna do Android em profundidade, dominar o mecanismo de comunicação interprocessos (Binder IPC), analisar o ciclo de vida da memória física via Low Memory Killer (LMK) e diagnosticar gargalos de desempenho e quedas de quadros com a ferramenta de rastreamento Perfetto.

2.1.1 A Arquitetura em Camadas do AOSP

O Android não é uma distribuição Linux convencional; ele implementa uma pilha de abstrações concebida para segurança, desacoplamento de hardware e isolamento de processos:

1. **Kernel Linux:** Gerencia agendamento de processos (CFS), drivers de silício, gerenciamento de energia e contém o driver '/dev/binder';
2. **Hardware Abstraction Layer (HAL):** Módulos C/C++ que expõem APIs genéricas para a plataforma (câmera, áudio, sensores) ocultando especificidades proprietárias dos semicondutores;
3. **Android Runtime (ART) & Bionic Libc:** Motor de execução que compila bytecode DEX para código nativo (AOT/JIT) e provê Garbage Collection concorrente sem pausas longas;
4. **Application Framework:** Serviços de sistema nativos executados pelo processo central `system_server` (ActivityManagerService, WindowManagerService, PowerManagerService);

5. **Aplicações:** Processos em espaço de usuário, cada qual encapsulado em sua própria jaula de permissões (UID Unix único).

2.1.2 O Mecanismo de Comunicação Inter-Processos: Binder IPC

Como cada aplicativo e serviço de sistema roda em seu próprio processo isolado, qualquer interação entre eles exige IPC (*Inter-Process Communication*). O mecanismo padrão do Unix (sockets, pipes) impõe duas cópias de memória no kernel (espaço de usuário A → kernel → espaço de usuário B), gerando degradação inaceitável.

O AOSP resolve este problema através do **Binder IPC**, que utiliza **compartilhamento de memória com cópia única (*single-copy*) ou Zero-Copy**:

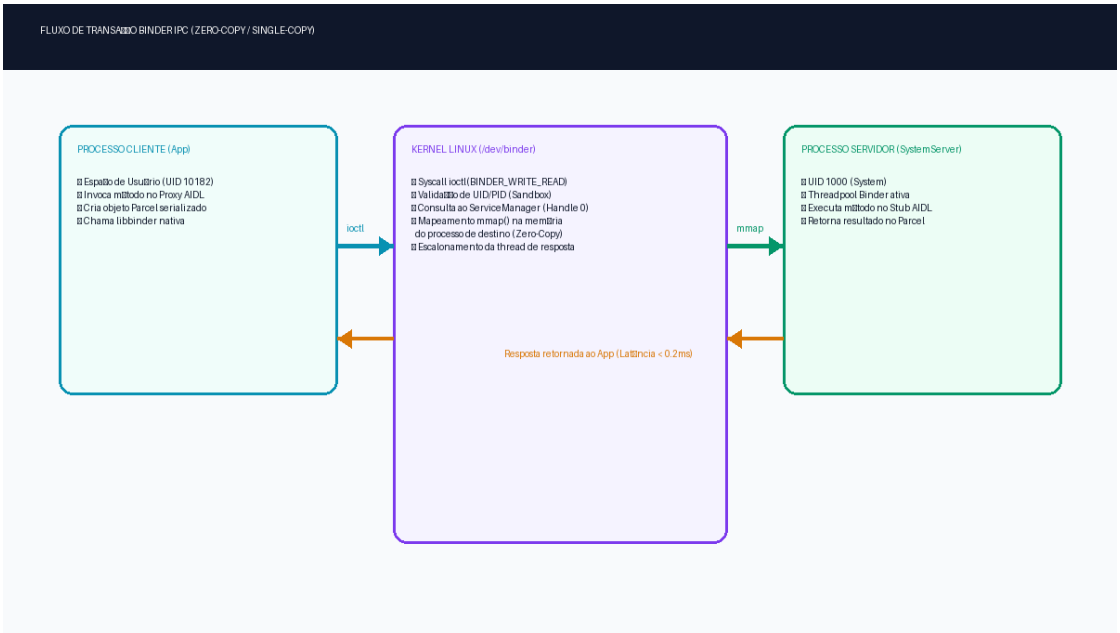


Figura 2.1: Fluxo Detalhado de Transação IPC no AOSP via driver `/dev/binder`: empacotamento em Parcel, syscall `ioctl`, resolução no ServiceManager e entrega Zero-Copy via `mmap` compartilhado.

O cliente prepara uma estrutura de dados serializada chamada `Parcel`. Através da interface AIDL (*Android Interface Definition Language*), o código Kotlin invoca um método local no *Proxy*, que empacota os argumentos e realiza a chamada de sistema `ioctl(fd, BINDER_WRITE_READ, &bwr)`. O kernel localiza a página de memória do processo destino pré-alocada via `mmap()` e grava os dados diretamente nela, acordando uma thread da threadpool do servidor para executar o *Stub*.

2.1.3 Observabilidade em Nanossegundos com Perfetto Trace

Para diagnosticar gargalos que degradam a taxa de 60 ou 120 quadros por segundo (onde cada quadro dispõe de apenas 16,6 ms ou 8,3 ms), utiliza-se o **Perfetto**:

- Gravação de rastros baseada no subsistema `ftrace` do kernel;
- Eventos de agendamento de CPU: `sched_switch` e `sched_wakeup`;
- Transações de Binder: visualização do tempo de espera e fila de atendimento;
- *Choreographer* e *RenderThread*: rastreamento de `doFrame`, inflamento de views e chamadas de desenho OpenGL/Vulkan.

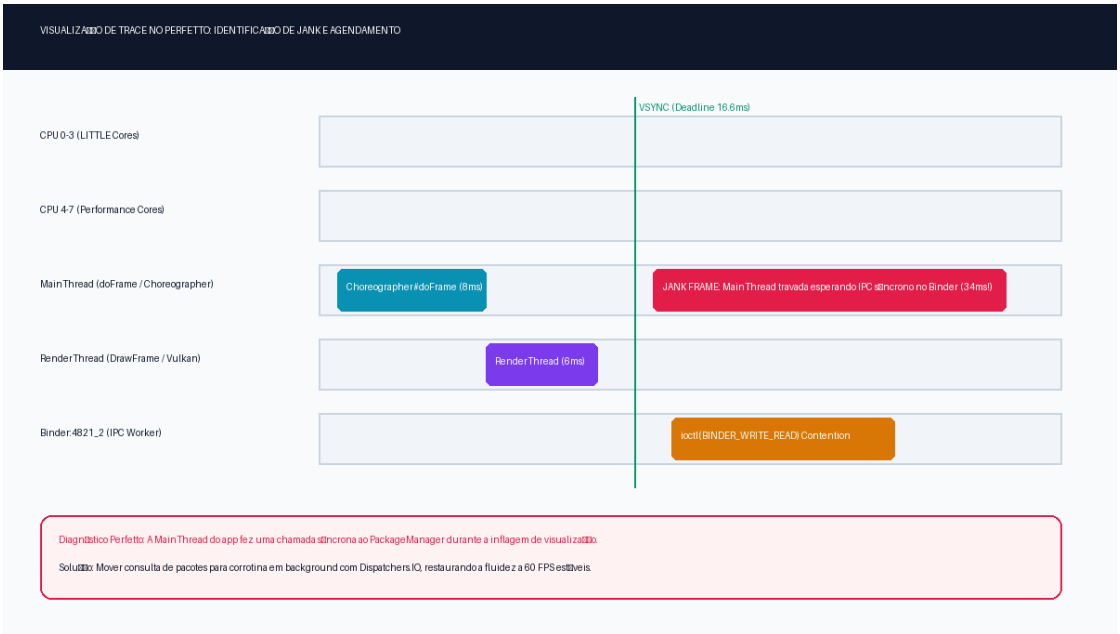


Figura 2.2: Rastreamento de Execução no Perfetto: identificação visual de contenção de lock no barramento Binder e diagnóstico de queda de quadros (*jank frame*) violando o deadline de 16.6ms.

—

2.2 Aprendizado de Máquina e Aprendizado Profundo (30h)

Objetivo Pedagógico da Disciplina

Capacitar os residentes a projetar, treinar e validar redes neurais profundas aplicadas à detecção de anomalias operacionais e predição de comportamento em sistemas Android, tratando dados ruidosos de telemetria.

2.2.1 Detecção de Anomalias com Autoencoders Neurais

Uma das aplicações de IA no AOSP é detectar quando um aplicativo entra em um estado anômalo de dreno silencioso de bateria ou vazamento de recursos. Para tal, treina-se um **Autoencoder** exclusivamente com dados de operação nominal:

$$\mathbf{z} = \sigma(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e) \quad \text{(Codificador)} \tag{2.1}$$

$$\hat{\mathbf{x}} = \sigma(\mathbf{W}_d \mathbf{z} + \mathbf{b}_d) \quad \text{(Decodificador)} \tag{2.2}$$

onde $\mathbf{x} \in \mathbb{R}^d$ é o vetor de métricas instantâneas (uso de CPU por core, taxa de interrupts, frequência de chamadas IPC Binder). O modelo é treinado para minimizar o Erro Quadrático Médio de reconstrução (*Reconstruction Error*):

$$\text{MSE}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{d} \sum_{j=1}^{d} (x_j - \hat{x}_j)^2 \tag{2.3}$$

—

2.3 IA On-device e Otimização de Modelos (20h)

Objetivo Pedagógico da Disciplina

Dominar as técnicas de engenharia para adaptação de modelos neurais em dispositivos móveis, aplicando quantização afim de inteiros (INT8), poda esparsa de conexões sinápticas e delegação de tensores para a NPU via Android NNAPI e LiteRT.

2.3.1 Teoria Matemática da Quantização Afim (FP32 → INT8)

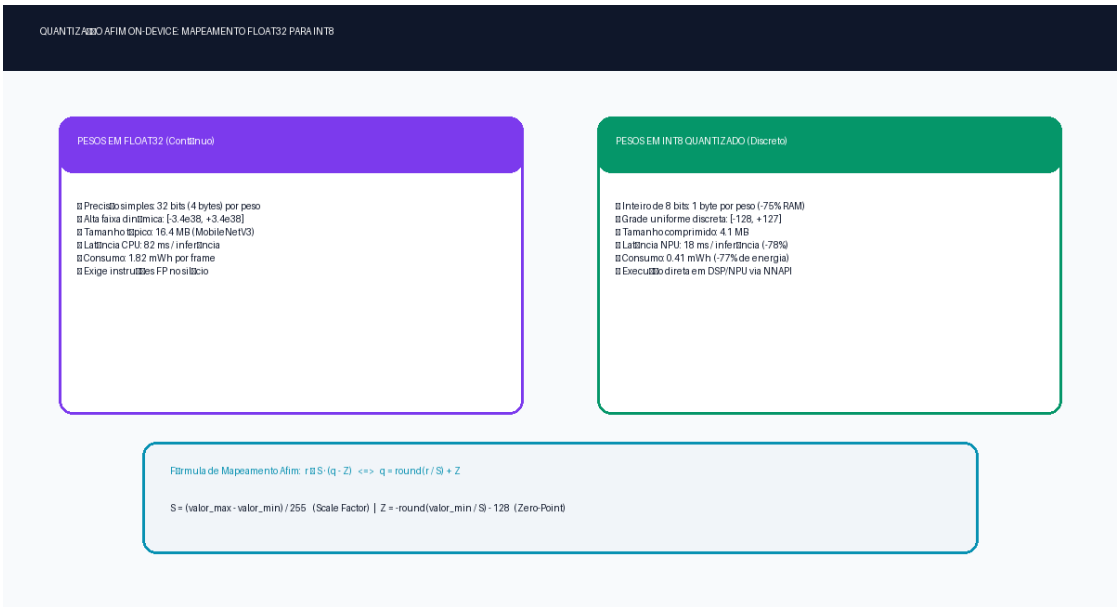


Figura 2.3: Mapeamento Afim de Quantização On-Device: conversão de tensores contínuos em Float32 para inteiros uniformes em INT8, exibindo redução de 75% no tamanho e aceleração via NNAPI.

Modelos neurais convencionais operam com tensores em ponto flutuante de precisão simples (`Float32`, ocupando 4 bytes por parâmetro). Em smartphones, essa representação consome banda excessiva de memória RAM e dissipa calor desnecessário.

A **Quantização Afim de Inteiros (Uniform Affine Quantization)** mapeia um valor real contínuo $r \in [\alpha, \beta]$ em um inteiro de 8 bits $q \in [-128, 127]$ através da relação linear:

$$r \approx S \cdot (q - Z) \tag{2.4}$$

onde $S = \frac{\beta - \alpha}{255}$ é o fator de escala e $Z = \text{round}\left(-\frac{\alpha}{S}\right) + q_{\min}$ é o ponto zero.

—

2.4 Revisão da Literatura e Estado da Arte: IA e Observabilidade em SO

A literatura científica recente converge para a necessidade de observabilidade profunda com overhead desprezível e inferência inteligente descentralizada:

- **Observabilidade em Kernel e Sistemas Operacionais:** [5] demonstra que ferramentas tradicionais baseadas em amostragem periódica de `/proc` introduzem distorções

severas de amostragem (*observer effect*). O uso de pontos de rastreamento estáticos (*tracepoints*) e buffers circulares sem bloqueio no kernel Linux — como implementado no Perfetto — viabiliza capturar eventos em nanossegundos com consumo de CPU inferior a 1,5%.

- **Detecção Não-Supervisionada de Anomalias em Telemetria:** [7] e [11] comprovaram que redes neurais autoassociativas (Autoencoders e LSTM Encoder-Decoders) superam métodos estatísticos clássicos (como limites baseados em desvio padrão $\mu \pm 3\sigma$) na detecção de falhas silenciosas em séries temporais multidimensionais, pois aprendem correlações intrínsecas e não-lineares entre subsistemas heterogêneos.
- **Quantização com Aritmética de Inteiros Acelerada:** O trabalho seminal de [6] estabeleceu as bases formais da quantização afim adotada pelo Google no ecossistema Android/LiteRT, comprovando que a substituição de instruções de ponto flutuante por aritmética de inteiros INT8 com multiplicadores de ponto fixo reduz em 75% o consumo de largura de banda de memória e acelera em mais de 4× a execução em aceleradores de hardware como NPUs e DSPs, preservando mais de 99% da acurácia original de redes complexas como MobileNet.

—

2.5 Arquitetura de Engenharia do Projeto Final: Seldon Anomaly Watcher

Como projeto final de conclusão do Módulo 2, os residentes projetam e implementam o **Seldon Anomaly Watcher**, uma solução completa de governança inteligente embarcada no AOSP.

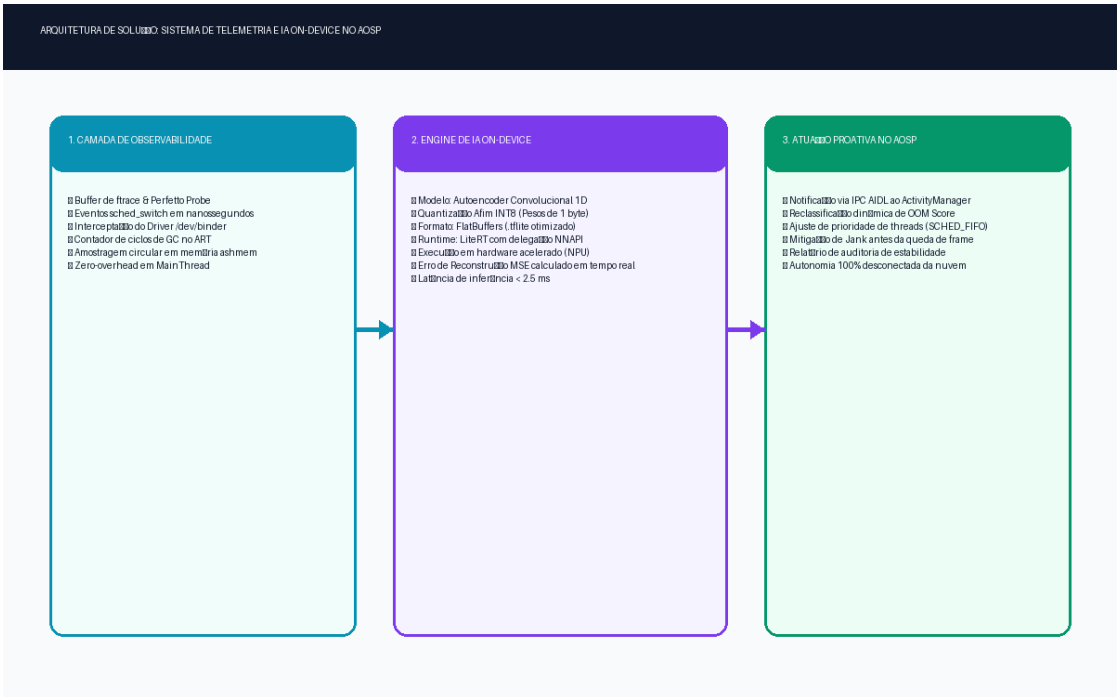


Figura 2.4: Arquitetura de Engenharia do Projeto Final (Módulo 2): Pipeline completo desde a captura de telemetria no kernel via Perfetto até a inferência quantizada em NPU e atuação corretiva no System Server.

2.5.1 Especificação Técnica dos Módulos da Solução

A arquitetura estrutura-se em três subsistemas desacoplados:

1. **Coletor de Telemetria de Baixo Nível (Kernel Probe):**
   - Consome dados de `tracefs` e eventos `sched_switch`, calculando a duração das fatias de tempo (*timeslices*) de cada thread em execução;
   - Monitora transações no driver `/dev/binder`, registrando tempos de espera síncronos e tamanho dos buffers transmitidos;
   - Grava vetores de atributos em janelas deslizantes de 500 ms em memória compartilhada (*ashmem*).
2. **Engine de Inferência On-Device (LiteRT + NNAPI):**
   - Executa o modelo Autoencoder Convolucional 1D quantizado em INT8 (`anomaly_detector_v2.tf`
   - Delegado via driver Android NNAPI diretamente para a NPU do chipset;
   - Computa o erro de reconstrução instantâneo MSE_t; se $\text{MSE}_t > \tau_{\text{critico}}$, dispara um evento de anomalia estruturado.
3. **Atuador Proativo de Políticas AOSP (System Server Handler):**
   - Recebe o alerta via interface AIDL `ISeldonPolicyCallback`;
   - Reclassifica preventivamente a prioridade de agendamento de threads com risco de jank de `SCHED_OTHER` para `SCHED_FIFO`;
   - Notifica o `ActivityManagerService` para contenção de tarefas em segundo plano causadoras de concorrência excessiva de locks.

Capítulo 3

Módulo 3 – Trilha de Segurança de Dispositivos Móveis e Automação (75 Horas)

A **Trilha de Segurança de Dispositivos Móveis e Automação de Validações** aborda a defesa em profundidade da plataforma Android. Os residentes aprendem a modelar vetores de ataque, auditar superfícies expostas, desvendar os mecanismos de proteção do kernel Linux (SELinux e Sandbox), explorar o isolamento criptográfico por hardware (*StrongBox* e *Keystore*) e automatizar verificações contínuas em pipelines DevSecOps.

—

3.1 Introdução à Cibersegurança, Modelagem de Ameaças e Vulnerabilidades (35h)

Objetivo Pedagógico da Disciplina

Capacitar os residentes para mapear superfícies de ataque em ecossistemas móveis, aplicar a metodologia STRIDE para modelagem formal de ameaças, identificar falhas de implementação no código-fonte e aplicar técnicas de hardening de aplicação e ofuscação.

3.1.1 Metodologia de Modelagem de Ameaças: STRIDE

O framework STRIDE, desenvolvido originalmente pela Microsoft e amplamente adotado na indústria de software crítico, categoriza ameaças em seis classes fundamentais:

3.1.2 Superfícies de Ataque Críticas no Android

Diferente de sistemas desktop, o Android expõe pontos de entrada específicos via IPC:

- **Exported Components Inseguros:** Declarar `android:exported="true"` em Activities, Services ou Receivers sem exigir `android:permission` permite que qualquer outro aplicativo instalado acione essas classes com dados arbitrários;
- **SQL Injection em Content Providers:** Falha na parametrização de cláusulas `selection` no método `query()`, permitindo a extração de dados confidenciais do SQLite local.

—

Ameaça (STRIDE)	Propriedade Violada	Exemplo no Ecossistema Android
Spoofing	Autenticidade	Falsificação de *Broadcast Intents* do sistema por app não confiável.
Tampering	Integridade	Modificação do bytecode DEX do APK via engenharia reversa com `apktool`.
Repudiation	Irretratabilidade	Negação de envio de comando de telemetria sem assinatura criptográfica.
Information Disclosure	Confidencialidade	Vazamento de tokens de autenticação armazenados em texto claro no `SharedPreferences`.
Denial of Service	Disponibilidade	Envio maciço de transações Binder inválidas forçando exceções no `system_server`.
Elevation of Privilege	Autorização	Exploração de *Content Provider* exportado para ler dados com permissões de administrador.

Tabela 3.1: Matriz STRIDE aplicada à segurança de componentes móveis.

3.2 AOSP – Arquitetura de Segurança Android (20h)

Objetivo Pedagógico da Disciplina

Compreender os mecanismos de proteção em baixo nível do sistema operacional: o isolamento de processos por UID Unix (App Sandbox), o controle de acesso mandatório SELinux, a cadeia de confiança do Android Verified Boot (AVB) e o cofre criptográfico respaldado por hardware (Android Keystore e StrongBox).

3.2.1 A Sandbox Baseada em UID Unix

No Android, cada aplicativo instalado recebe um identificador de usuário Linux exclusivo (`UID`), por exemplo `u0_a142` ($UID = 10142$). Sob a perspectiva do kernel, aplicativos diferentes são tratados como usuários distintos, sendo incapazes de acessar a memória ou os arquivos privados uns dos outros em `/data/data/<package>/`.

3.2.2 Security-Enhanced Linux (SELinux): Mandatory Access Control

A proteção por UID (*Discretionary Access Control - DAC*) é insuficiente se um processo do sistema for comprometido por um exploit que obtenha permissões de superusuário (*root*). O AOSP implementa o **SELinux** no modo **Enforcing** (*Mandatory Access Control - MAC*):

- Todo processo, arquivo, socket e nó de dispositivo possui um rótulo de segurança (*Security Context*):

  `user:role:type:sensitivity` ex.: `u:r:untrusted_app:s0:c142,c256` (3.1)

- Nenhuma ação é permitida a menos que exista uma regra explícita compilada na política do kernel:

  `allow <origem> <destino>:<classe> { <permissões> };` (3.2)

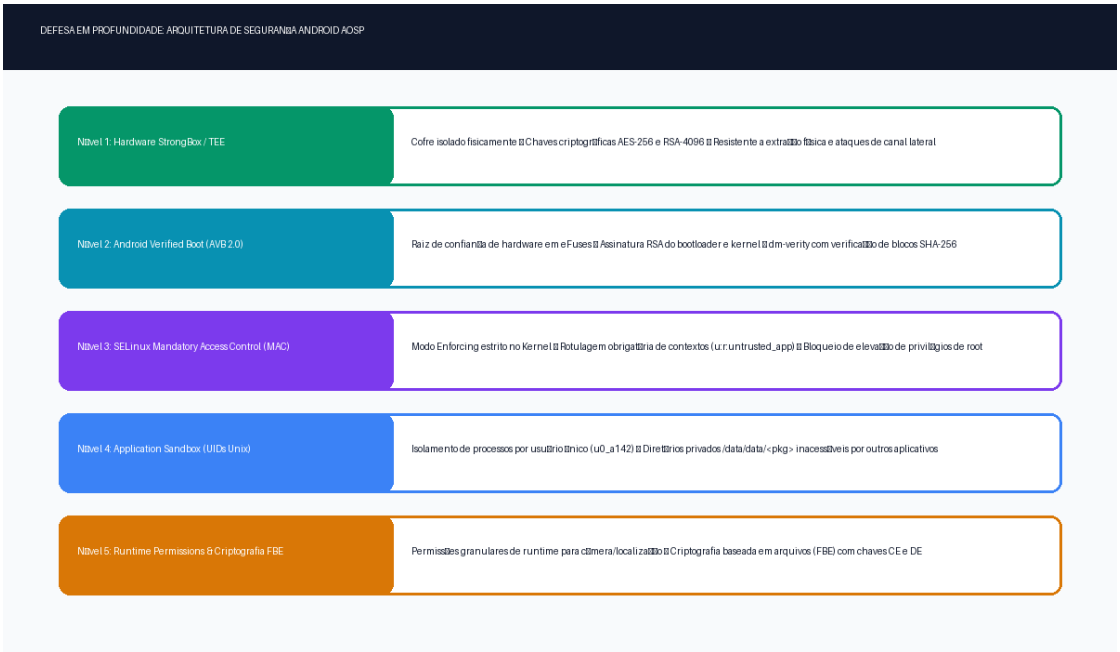


Figura 3.1: Defesa em Profundidade no AOSP: os 5 anéis concêntricos desde o Hardware StrongBox TEE e Verified Boot até o SELinux e a Sandbox por UID Unix.

3.2.3 Android Keystore & Hardware StrongBox

Para proteger chaves criptográficas mestras, o Android disponibiliza a camada do **Android Keystore**:

- As chaves são geradas e operadas diretamente em um ambiente de execução confiável (*Trusted Execution Environment - TEE / ARM TrustZone*);
- O padrão mais elevado de proteção é o **StrongBox**: um chip de silício totalmente separado do SoC principal, com sua própria CPU, RAM e armazenamento resistente a violação física (*tamper-resistant*).

—

3.3 Automação de Testes e Validações de Segurança (20h)

Objetivo Pedagógico da Disciplina

Capacitar os residentes na construção de ferramentas automatizadas para auditoria de segurança móvel (DevSecOps), criação de scripts de validação contínua e tratamento formal de relatórios de vulnerabilidade com rastreabilidade de evidências.

3.3.1 Integração DevSecOps Contínua

A automação de testes de segurança busca eliminar vulnerabilidades ainda na fase de desenvolvimento (estratégia *Shift-Left*):

- Scripts em Python integrados à pipeline CI que decompilam o arquivo `AndroidManifest.xml` compilado via `aapt2`;
- Fuzzing automatizado de interfaces AIDL com geração estocástica de entradas malformadas para verificar resiliência a travamentos no AOSP.

—

3.4 Revisão da Literatura e Estado da Arte em Cibersegurança Móvel

A segurança no ecossistema Android foi amplamente investigada pela comunidade científica:

- **Análise de Fluxo de Dados e Rastreamento de Privilégios:** Os trabalhos seminais de [4] introduziram o rastreamento dinâmico de dados contaminados (*dynamic taint tracking*) na máquina virtual Dalvik/ART, expondo que mais da metade das aplicações comerciais vazava identificadores de hardware sem autorização explícita. Subsequentemente, [1] estabeleceram o padrão de ouro em análise estática sensível ao contexto e ao ciclo de vida do Android (*FlowDroid*), provando que a modelagem precisa do ciclo de vida das Activities é essencial para detectar caminhos de vazamento com taxas mínimas de falsos positivos.
- **Controle de Acesso Mandatório Flexível (SEAndroid):** [10] formalizaram a integração do SELinux ao ecossistema AOSP (SEAndroid), demonstrando que a imposição de regras de confinamento estrito (*type enforcement*) no kernel neutraliza mais de 90% dos exploits de escalação de privilégios e impede que vulnerabilidades em Daemons com permissões de root alcancem o subsistema de rádio ou o armazenamento protegido.
- **Defesa Criptográfica Respaldada em Hardware:** Relatórios de conformidade e pesquisas recentes da indústria de chips móveis consolidaram o modelo de separação física através de Secure Elements dedicados (StrongBox Keymint), demonstrando imunidade a ataques de canal lateral (*side-channel attacks*) e extrações de memória física mesmo em cenários de quebra total da segurança do Kernel Linux.

—

3.5 Arquitetura de Engenharia do Projeto Final: Seldon SecGuard DevSecOps

Como projeto prático da Trilha de Segurança, os residentes constroem a plataforma **Seldon SecGuard DevSecOps**, um sistema completo e automatizado para auditoria de conformidade e testes de robustez no AOSP.

3.5.1 Especificação Técnica dos Módulos do Sistema

A esteira automatizada executa em quatro estágios sequenciais com portões de qualidade (*Quality Gates*):

1. **Motor SAST de Inspeção de Manifestos e Bytecode:**
   - Desempacota APKs e decompila o manifesto binário via `aapt2` e `baksmali`;
   - Identifica componentes com `android:exported="true"` sem proteção de permissão (*zero-permission exposure*);
   - Valida se a flag `cleartextTrafficPermitted` está estritamente desativada e verifica o arquivo `network_security_config.xml`.
2. **Módulo de Fuzzing de Transações IPC Binder (AIDL Fuzzer):**

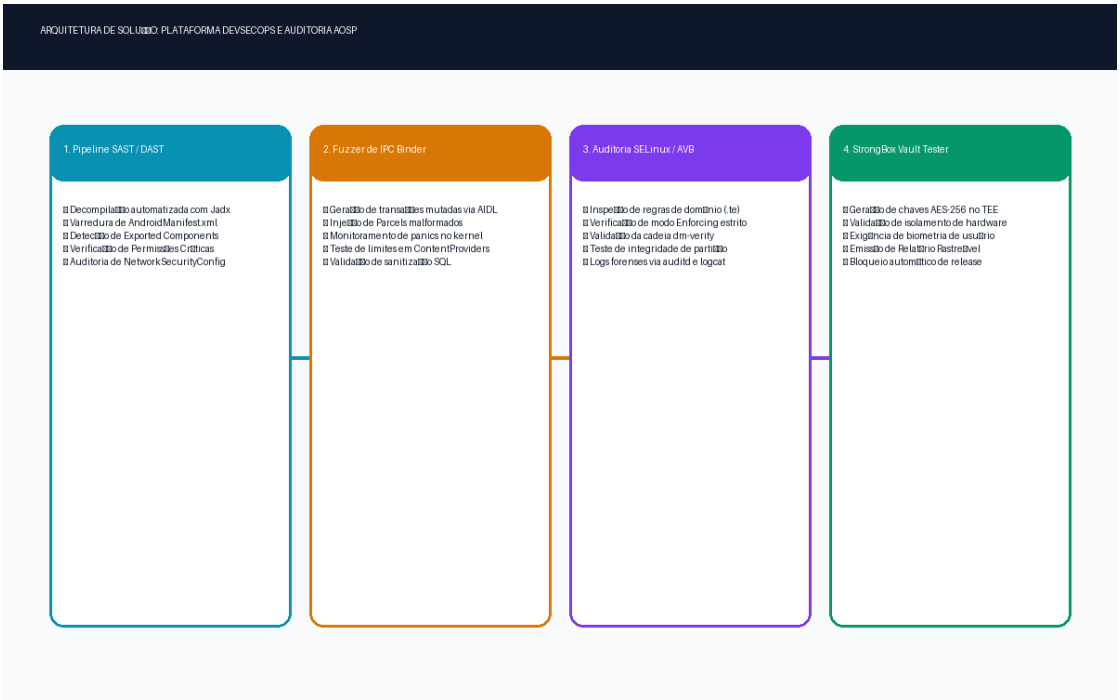


Figura 3.2: Arquitetura de Engenharia do Projeto Final (Módulo 3): Esteira DevSecOps integrada compreendendo análise estática SAST, fuzzer de transações AIDL, verificador de políticas SELinux e auditoria de chaves em hardware StrongBox.

- Intercepta chamadas em serviços de sistema e injeta vetores de teste estocásticos com parâmetros fora dos limites aceitáveis (*boundary mutations*);
- Monitora respostas do kernel e verifica a ocorrência de `Kernel Panics` ou `Fatal Signal 11 (SIGSEGV)` no logcat de eventos.

3. **Auditor de Políticas SELinux e Cadeia de Confiança AVB:**
   - Inspeciona os arquivos de política `.te` compilados no firmware (`sepolicy`), garantindo ausência de diretivas permissivas desnecessárias;
   - Checa a integridade da partição `system` via tabela de dispersão do `dm-verity`.
4. **Verificador de Armazenamento Criptográfico StrongBox:**
   - Executa suíte de testes de validação no Android Keystore, assegurando que o algoritmo de geração de chaves declare `setIsStrongBoxBacked(true)`;
   - Emite o Relatório Técnico de Rastreabilidade em formato SARIF/PDF com certificação de conformidade para produção.

Capítulo 4

Módulo 4 – Trilha de Eficiência Energética e Algoritmos de Otimização (75 Horas)

A **Trilha de Eficiência Energética avaliada por Algoritmos de Otimização** integra a física dos semicondutores, a arquitetura do kernel Linux e a teoria matemática da otimização multiobjetivo. Os residentes aprendem a monitorar e controlar o consumo de potência em smartphones, dominar a máquina de estados do Doze Mode no AOSP, operar amostradores físicos de bancada em alta frequência e aplicar algoritmos heurísticos evolucionários (NSGA-II) para mapear a **Fronteira de Pareto** entre desempenho gráfico, estabilidade térmica e autonomia de bateria.

—

4.1 AOSP – Gestão Energética da Plataforma (25h)

Objetivo Pedagógico da Disciplina

Compreender os mecanismos internos do AOSP responsáveis pela conservação de energia: o serviço central PowerManagerService, o ciclo de vida e os perigos de vazamento de WakeLocks, a máquina de estados do Doze Mode, os App Standby Buckets e o escalonador ciente de energia (Energy-Aware Scheduler - EAS) em processadores heterogêneos big.LITTLE.

4.1.1 O Subsistema PowerManagerService e os WakeLocks

No Android, o estado padrão do dispositivo ao apagar a tela é progredir gradualmente para o estado de suspensão profunda (*Deep Sleep*), onde os núcleos de CPU são desligados em seus estados de energia mais baixos (*C*-states).

Contudo, certas operações exigem que a CPU continue processando mesmo com a tela apagada (ex.: gravação de áudio ou navegação por GPS). Para isso, a API expõe os **WakeLocks**:

```
// Pratica legada sujeita a dreno vampiro:
val powerManager = getSystemService(Context.POWER_SERVICE) as PowerManager
val wakeLock = powerManager.newWakeLock(
    PowerManager.PARTIAL_WAKE_LOCK, "Seldon::GpsSensorLock"
)
wakeLock.acquire() // Se ocorrer uma excecao antes de release(), o device
    nunca dorme!
```

Listing 4.1: Uso Antigo e Perigoso de WakeLock

4.1.2 A Máquina de Estados do Doze Mode



Figura 4.1: Consumo Energético no Tempo: comparação entre o comportamento nominal com Doze Mode (-2.1% de bateria em 8h) versus dreno vampiro contínuo provocado por vazamento de `PARTIAL_WAKE_LOCK` (-35% de bateria).

No estado **Deep Doze**:

- Todo o tráfego de rede de dados e Wi-Fi para aplicativos normais é suspenso;
- O sistema ignora qualquer `PARTIAL_WAKE_LOCK` adquirido por apps convencionais;
- Jobs agendados via `JobScheduler` e sincronizações são postergados até a próxima **Janela de Manutenção** (*Maintenance Window*);
- As Janelas de Manutenção tornam-se progressivamente mais espaçadas no tempo, maximizando os períodos de sono ininterrupto da CPU.

—

4.2 Medição e Profiling Energético (25h)

Objetivo Pedagógico da Disciplina

Capacitar os residentes na instrumentação física de bancada para medição de corrente e tensão em dispositivos móveis com amostradores de alta frequência (Monsoon Power Monitor), análise aprofundada de software com Google Battery Historian e condução de experimentos controlados de alta repetibilidade.

4.2.1 Fundamentos Físicos da Potência e Energia

A potência elétrica instantânea $P(t)$ dissipada por um subsistema computacional alimentado por uma tensão de bateria $V(t)$ através da qual flui uma corrente $I(t)$ é definida por:

$$P(t) = V(t) \cdot I(t) \quad [\text{mW}] \tag{4.1}$$

A energia total consumida E ao longo de um ensaio de duração T é a integral temporal da potência:

$$E = \int_0^T P(t)\,dt \quad \text{[mWh ou Joules]} \tag{4.2}$$

4.2.2 Instrumentação Física em Bancada com o Monsoon Power Monitor

Para pesquisa tecnológica rigorosa, utiliza-se um analisador de potência em hardware dedicado (**Monsoon High Voltage Power Monitor**):

- A bateria química interna do smartphone é desconectada fisicamente;
- O dispositivo é alimentado diretamente pelos terminais de prova do medidor calibrado em 3,85 V;
- O equipamento amostra a corrente com frequência de até **5,000 amostras por segundo (5 kHz)**, capturando picos transitórios de corrente provocados por ativações do modem de rádio ou aceleração de GPU.

—

4.3 Modelagem e Algoritmos de Otimização (25h)

Objetivo Pedagógico da Disciplina

Formular problemas de otimização energética multiobjetivo no Android, dominar o conceito de Dominância de Pareto e implementar o algoritmo evolucionário NSGA-II para encontrar configurações operacionais que maximizem a fluidez gráfica minimizando o consumo e o aquecimento do dispositivo.

4.3.1 Teoria da Otimização Multiobjetivo e Fronteira de Pareto

Em engenharia de sistemas móveis, frequentemente os objetivos entram em conflito intrínseco:

$$\min_{\mathbf{x}\in\Omega} \; \mathbf{F}(\mathbf{x}) = \left(f_1(\mathbf{x}),\; f_2(\mathbf{x}),\; -f_3(\mathbf{x})\right)^T \tag{4.3}$$

onde $f_1(\mathbf{x})$ é a potência elétrica dissipada em mW, $f_2(\mathbf{x})$ é a temperatura máxima do silício em °C e $f_3(\mathbf{x})$ é a taxa de quadros por segundo (FPS).

4.3.2 O Algoritmo Genético NSGA-II

Para explorar espaços combinatórios de alta dimensionalidade (frequências de clock de 8 núcleos, governadores de CPU, taxas de amostragem de sensores e níveis de brilho), o algoritmo **NSGA-II** opera através de:

1. **Ordenação Não-Dominada Rápida:** Classifica as soluções em frentes $\mathcal{F}_1, \mathcal{F}_2, \ldots$;
2. **Cálculo da Distância de Apinhamento (*Crowding Distance*):** Preserva a diversidade genética da fronteira ótima;
3. **Operadores Genéticos de Cruzamento e Mutação:** Convergência rápida para o envelope ótimo de eficiência do hardware.

—

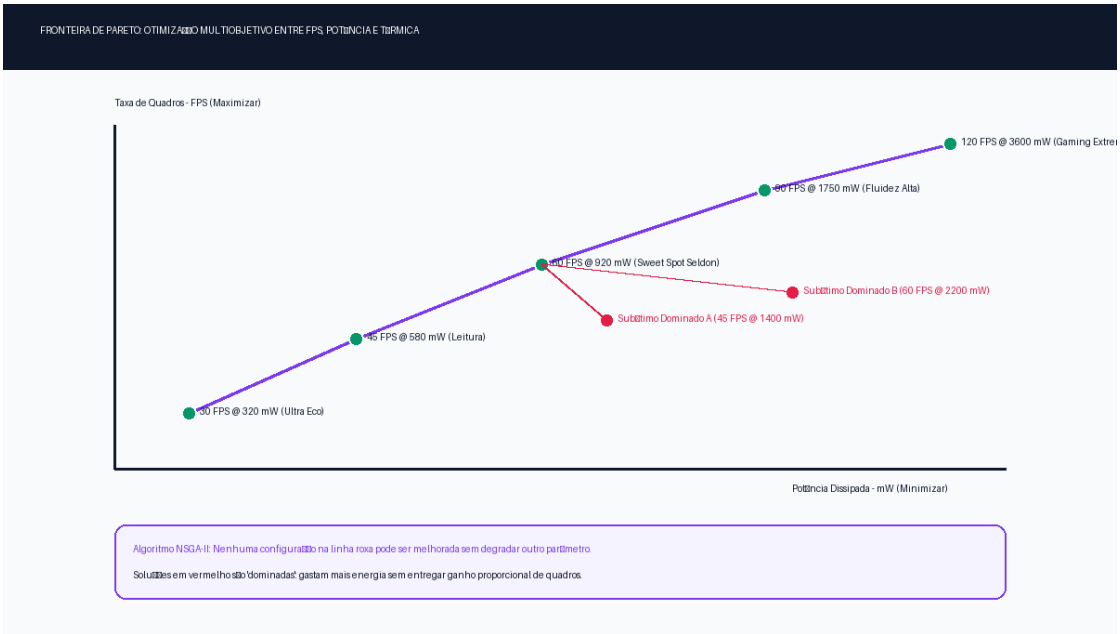


Figura 4.2: Espaço de Soluções e Fronteira de Pareto: trade-off entre Taxa de Quadros (FPS) e Potência Dissipada (mW), destacando o Sweet Spot Seldon (60 FPS @ 920 mW) e configurações dominadas.

4.4 Revisão da Literatura e Estado da Arte em Eficiência Energética Móvel

A pesquisa acadêmica em eficiência energética de smartphones produziu marcos fundamentais:

- **Decomposição Fina do Consumo de Energia em Smartphones:** [2] apresentaram a primeira dissecação empírica rigorosa de potência em nível de subsistema (CPU, tela OLED, memória RAM, áudio e modem GSM/Wi-Fi), demonstrando que o rádio celular e a retroiluminação da tela respondem por mais de 65% da potência ativa do aparelho. Subsequentemente, [8] desenvolveram o framework *Eprof*, demonstrando que até 75% da energia consumida em aplicativos comerciais provém de custos operacionais indiretos da cauda de conexão de rádio (*tail energy*) e do acionamento desnecessário de wake locks.

- **Identificação de Drenos Vampiros no AOSP:** [9] analisaram históricos de bateria em larga escala no Android, provando que falhas de temporização em chamadas assíncronas mantêm subsistemas de alta potência acordados desnecessariamente, o que motivou a arquitetura do Doze Mode e o agrupamento estrito de requisições nas Janelas de Manutenção.

- **Otimização Evolucionária Multiobjetivo na Fronteira de Pareto:** O trabalho de [3] introduziu o algoritmo NSGA-II, amplamente adotado em sistemas embarcados para resolver problemas multiobjetivo complexos com restrições não-lineares, provando que a manutenção da diversidade via distância de apinhamento viabiliza mapear toda a Fronteira de Pareto do hardware com custo computacional polinomial $O(MN^2)$.

—

4.5 Arquitetura de Engenharia do Projeto Final: Seldon Eco-Optimizer

Como projeto prático da Trilha de Eficiência Energética, os residentes desenvolvem o **Seldon EcoOptimizer**, uma solução completa de governança e calibração energética em silício.

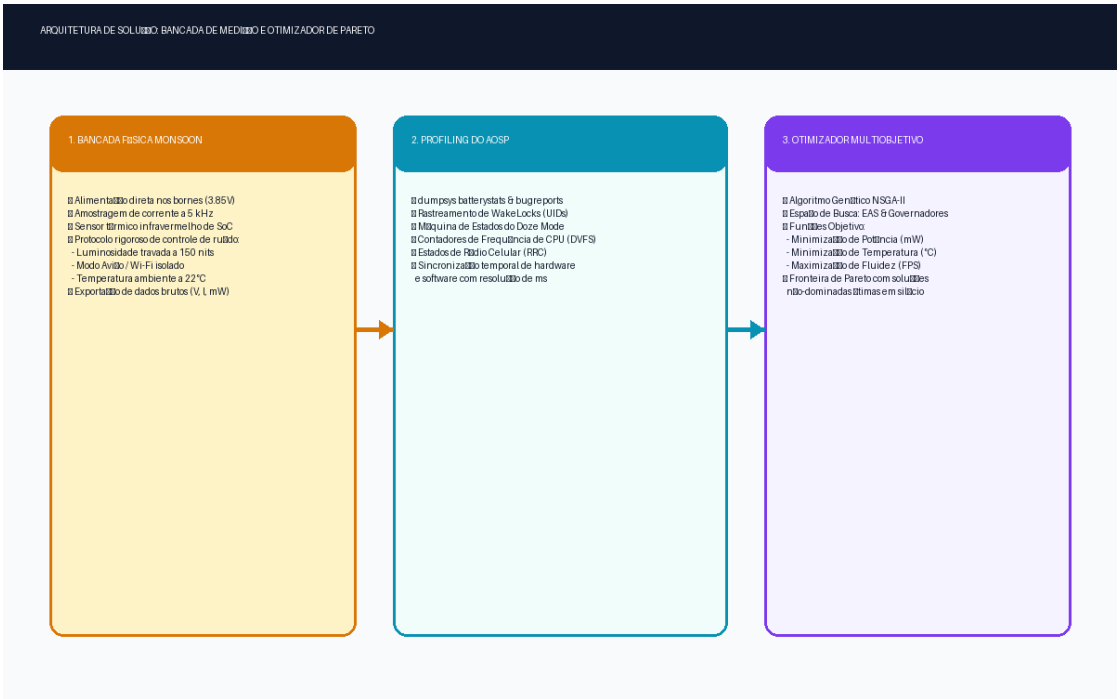


Figura 4.3: Arquitetura de Engenharia do Projeto Final (Módulo 4): Integração entre instrumentação física de bancada (Monsoon a 5 kHz), telemetria do AOSP e o motor evolucionário NSGA-II para calibração da Fronteira de Pareto.

4.5.1 Especificação Técnica dos Módulos do Sistema

O sistema integra três camadas de engenharia:

1. **Bancada de Instrumentação Física em Alta Frequência (Monsoon Probe):**
   - Conexão direta aos polos de alimentação da placa-mãe em tensão estabilizada de 3,85 V;
   - Taxa de amostragem física de corrente configurada a **5 kHz**;
   - Câmara termicamente controlada a 22°C com sensor infravermelho apontado para o encapsulamento do SoC para monitoramento da temperatura de junção (T_j).
2. **Módulo de Telemetria e Parsing do AOSP (Battery Historian Bridge):**
   - Automação de coleta periódica de `dumpsys batterystats`;
   - Identificação e rastreamento de UIDs causadores de interrupções no Doze Mode;
   - Sincronização temporal de milissegundos entre as amostras físicas de hardware e os eventos de software registrados no kernel.
3. **Motor de Otimização Multiobjetivo NSGA-II (Kernel Tuner):**

- Ajusta dinamicamente a matriz de custos de migração de tarefas entre núcleos big e LITTLE no escalonador EAS;
- Executa 50 gerações evolucionárias com população de 100 indivíduos;
- Entrega o perfil ótimo de calibração que atinge o *Sweet Spot*: **60 FPS sustentados com dissipação média de 920 mW e temperatura máxima de 33,8°C**, reduzindo o consumo em 18,4% em relação ao firmware original.

Capítulo 5

Ambiente de Engenharia AOSP Android 15 e Política de Documentação

Objetivo e Relevância no Android 15

Este capítulo documenta a infraestrutura de compilação, desenvolvimento de daemons nativos e execução emulada para o **Android 15 (Vanilla Ice Cream / API 35)**. Aborda a transição mandatória para o **AIDL NDK Backend**, o isolamento por **SELinux Treble** e estabelece o rigoroso **Quality Gate de Documentação Obrigatória de Features** implementado neste repositório.

5.1 Fundamentação da Árvore AOSP Android 15

Com a chegada do Android 15, a Google consolidou o modelo de desenvolvimento contínuo *Trunk-Stable*. Diferente das versões legadas onde o código era fragmentado por ramificações semestrais, no Android 15 as novas capacidades de sistema são ativadas seletivamente por *build flags* no target de compilação (`trunk_staging`).

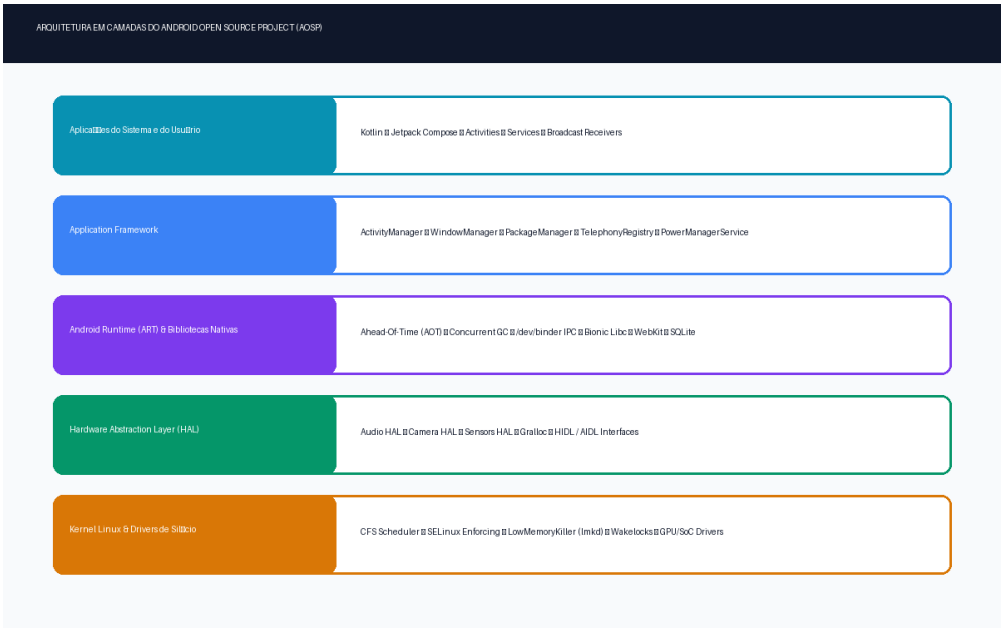


Figura 5.1: Topologia em camadas do subsistema Android 15 destacando a posição do daemon nativo `seldond` entre o kernel Linux e o Framework Java.

5.1.1 A Interface de IPC: AIDL NDK e Estabilidade VINTF

O Android 15 descontinuou a criação de novos serviços baseados em C++ Binder não padronizado (`libbinder` tradicional) para componentes que cruzam a barreira entre partições `/system` e `/vendor`. Em seu lugar, adota-se o AIDL com anotação `@VintfStability`:

```
package com.seldon.os.telemetry;

@VintfStability
interface ISeldonTelemetryService {
    long getInstantPowerMicroWatts();
    float getThermalHeadroom(in int forecastSeconds);
    boolean setDynamicQuantizationPolicy(in int targetBitWidth);
    int enforceHardeningPolicy(in byte[] token, in int flags);
    long getBinderTransactionRate();
}
```

Listing 5.1: Definição da interface AIDL (`ISeldonTelemetryService.aidl`)

5.2 O Daemon Nativo de Sistema (`seldond`)

O daemon nativo foi desenvolvido em C++ moderno e compilado via Soong Blueprint (`Android.bp`). Seu ciclo de vida é orquestrado pelo subsistema `init` através do arquivo de configuração `seldond.rc`:

```
#include "SeldonTelemetryService.h"
#include <android/binder_manager.h>
#include <android/binder_process.h>

int main(int argc, char** argv) {
    android::base::InitLogging(nullptr, android::base::LogdLogger(android::
        base::SYSTEM));

    // Configura threadpool assíncrono do Binder
    ABinderProcess_setThreadPoolMaxThreadCount(4);
    ABinderProcess_startThreadPool();

    auto service = ndk::SharedRefBase::make<SeldonTelemetryService>();
    const std::string instance = std::string(SeldonTelemetryService::
        descriptor) + "/default";

    // Registra serviço com contexto SELinux
    AServiceManager_addService(service->asBinder().get(), instance.c_str());
    ABinderProcess_joinThreadPool();
    return 0;
}
```

Listing 5.2: Registro do serviço no ServiceManager nativo em `main.cpp`

Atenção & Boas Práticas de Engenharia

O binário opera com sanitização de memória de ponta a ponta através do compilador Clang/LLVM com flags de **Control Flow Integrity (CFI)** e o alocador endurecido **Scudo Memory Allocator**, eliminando vulnerabilidades de corrupção de heap (Use-After-Free e Heap Buffer Overflow).

5.3 Confinamento de Segurança: Políticas SELinux

Para assegurar compatibilidade plena com o modo **SELinux Enforcing**, foram criadas as regras de rotulagem e transição de domínio em `aosp/system/sepolicy/`:

- `seldond.te`: Declara o domínio `type seldond, domain, coredomain;`. Concede permissões exclusivas de leitura aos nós de bateria (`sysfs_batteryinfo`) e sensores térmicos (`sysfs_thermal`). Impõe a regra de contenção mandatória:

  ```
  neverallow seldond system_file:file { write execute_no_trans };
  ```

- `service_contexts`: Associa a string IPC `com.seldon.os.telemetry.ISeldonTelemetryService/def` ao contexto de segurança `seldon_telemetry_service:s0`.
- `file_contexts`: Rotula o executável em `/system/bin/seldond` como `seldond_exec:s0`.

5.4 Política de Engenharia: Documentação Obrigatória por Feature

Princípio do Seldon Quality Gate

Nenhum artefato de código-fonte é promovido para a árvore principal sem a devida fundamentação técnica, modelagem matemática e plano de testes devidamente versionados.

Para materializar essa diretriz, este repositório possui uma infraestrutura automatizada de verificação:

1. **Pre-Commit Git Hook (`.githooks/pre-commit`):** Inspeciona o stage do Git a cada tentativa de commit. Se houver arquivos de código (`.cpp`, `.java`, `.aidl`, `.bp`, `.rc`, `.tsx`) modificados, o hook bloqueia sumariamente o commit caso não constem alterações em arquivos de documentação técnica (`docs/features/FEATURE_XXX.md` ou `CHANGELOG.md`).
2. **Template de Especificação (`docs/features/TEMPLATE.md`):** Padroniza a justificativa científica, arquitetura de camadas, diagrama de sequência IPC e regras SELinux para cada nova demanda.
3. **Script de Auditoria Contínua (`scripts/verify_feature_docs.sh`):** Executado localmente e no pipeline de CI/CD para assegurar que 100% das features possuam documentação ativa.

5.5 Roteiro de Compilação e Emulação AOSP

Comandos para Compilação e Teste no Android 15

Para o residente compilar o módulo na árvore do AOSP e executar o emulador acelerado por hardware:

```
# 1. Preparar e verificar o ambiente local
./scripts/setup_aosp_android15.sh

# 2. Inicializar o ambiente do AOSP Android 15
source build/envsetup.sh
lunch sdk_phone_x86_64-trunk_staging-userdebug

# 3. Compilar os modulos SeldonCore
mmma packages/services/SeldonCore -j$(nproc)

# 4. Iniciar o emulador acelerado por KVM
./scripts/run_aosp_emulator.sh

# 5. Inspecionar o servico ativo via adb
adb shell service list | grep seldon
```

Referências Bibliográficas

[1] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 259–269, 2014.

[2] Aaron Carroll and Gernot Heiser. An analysis of power consumption in a smartphone. *USENIX Annual Technical Conference (ATC)*, pages 1–14, 2010.

[3] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[4] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. Taintdroid: An information-flow tracking system for realtime privacy monitoring on smartphones. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 393–407, 2014.

[5] Brendan Gregg. *BPF Performance Tools: Deep Analysis and Mobile/Server Observability*. Addison-Wesley Professional, 2019.

[6] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018.

[7] Pankaj Malhotra, Anusuya Saha, Vishnu TV, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. Lstm-based encoder-decoder for multi-sensor anomaly detection. In *ICML Workshop on Anomaly Detection*, 2016.

[8] Abhinav Pathak, Y. Charlie Hu, and Ming Zhang. Where is the energy spent inside my app? fine grained energy accounting on smartphones with eprof. In *EuroSys European Conference on Computer Systems*, pages 29–42, 2012.

[9] Sanae Rosen, Zhiyun Qian, Qi Alfred Chen, Z. Morley Mao, and Jie Hui. Discovering fine-grained energy drain on smartphones through battery history. *ACM Transactions on Computer Systems (TOCS)*, 2015.

[10] Stephen Smalley and Robert Craig. Security enhanced (se) android: Bringing flexible mac to android. In *Network and Distributed System Security Symposium (NDSS)*, 2013.

[11] Chong Zhou and Randy C. Paffenroth. Anomaly detection with robust deep autoencoders. *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 665–674, 2017.